Task 1 – QA Report

Automation & QA Developer Take-Home Assessment

Tester: Nitin Maurya | Application: RealWorld Demo | Date: 29 September 2026

1. Test Summary

I tested the hosted RealWorld application at demo.realworld.show. I focused on the main user flows requested in the assessment and then added validation and edge-case checks. The testing included registration, login, article creation/edit/delete, logout, profile settings, comments, favorites, follows, and basic input/security checks.

2. Key Findings

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Invalid email accepted during registration	1. Open the Sign Up page. 2. Enter a syntactically invalid email value such as abc. 3. Enter a unique username and a password. 4. Submit the registration form.	**Expected:** The form should reject an invalid email format and show a clear validation message. **Actual:** Registration was successful even though the email value was not in a valid email format.	Medium	Email format validation appears to be missing or insufficient on the registration flow. The backend also appears to accept the value instead of rejecting it.
2	Extremely short password accepted during registration	1. Open the Sign Up page. 2. Enter a valid/unique username and email. 3. Enter 1 as the password. 4. Submit the registration form.	**Expected:** The application should enforce a reasonable password validation rule or at least reject an extremely short password with a clear message. **Actual:** The account was created successfully with the password `1`, with no validation warning.	Medium	The registration flow appears not to enforce a minimum password length/strength requirement. The exact intended minimum is not specified by the assignment, so the finding is based on accepting an extremely short password.
3	Duplicate username/email allowed during registration	1. Create an account using a username and email. 2. Attempt another registration using an already-used username and/or already-used email. 3. Submit the second registration.	**Expected:** The application should reject duplicate identity fields and clearly indicate that the username or email is already in use. **Actual:** Duplicate username and duplicate email values were accepted during registration and the registration request succeeded.	High	The registration flow appears to lack effective uniqueness validation and/or a backend uniqueness constraint for username and email. This should be enforced server-side so it cannot be bypassed by a client.
4	Whitespace-only article content accepted	1. Open New Article. 2. Enter one space ( ) in the Title field. 3. Enter one space ( ) in the Description field. 4. Enter one space ( ) in the Body field. 5. Submit the article.	**Expected:** Whitespace-only values should be treated as empty input and rejected with validation messages. **Actual:** The article was created successfully with whitespace-only values and no validation error was shown.	Medium	Input validation appears to check whether the fields contain characters without first trimming/normalizing whitespace. A server-side validation check should also reject values that become empty after trimming.
5	Invalid profile image URL accepted and persisted	1. Open Settings/Profile Settings. 2. Enter 2 in the Profile Image URL field. 3. Save/Update the profile. 4. Refresh the page and check the saved value.	**Expected:** The field should validate that the supplied value is a valid URL, or reject it with a clear validation message. **Actual:** The profile update succeeded with `2` as the image URL, and the value remained saved after refreshing the page.	Medium	The profile update flow appears not to validate the image field as a URL before saving it. Server-side URL validation would prevent invalid values from being persisted.

3. Root-Cause Analysis – Whitespace-only Article Content

The clearest root-cause example is the whitespace-only article validation issue.

The application accepted a single space in the title, description, and body and created the article without showing a validation error.

This suggests that the current validation is checking for a non-empty string without normalizing the input first.

A value such as ` ` is technically non-empty as a raw string, but it has no meaningful content after whitespace is removed.

The fix should trim the input before performing required-field validation, and the same rule should also be enforced on the server/API layer.

The backend should reject a value when the trimmed result is empty, rather than relying only on the browser-side form validation.

The UI should then show a field-level validation message so the user knows what needs to be corrected.

I would also add automated tests for empty strings, one or more spaces, tabs, and normal text to prevent the issue from returning.

4. Additional Passing Checks

• Sign-up, login, article create/edit/delete, and logout flows were exercised.

• Empty article fields were rejected.

• Empty comments were rejected.

• Favorite/unfavorite worked as expected.

• Follow/unfollow worked as expected.

• The XSS/HTML test did not execute JavaScript; the supplied script was rendered as text.

5. Notes

The five findings above are based on behavior reproduced during testing. Closely related whitespace validation behavior was also observed in comments, but it is not counted as a separate main finding to avoid double-counting the same validation weakness.

Severity is based on the practical impact observed during testing. The exact password minimum and URL policy are not specified in the assessment, so the report does not assume a specific required password length or image-host policy.